Pertemuan 7

Bottom Navigation · ListView · Custom Adapter

YANG AKAN KITA KERJAKAN

Di pertemuan ini kita akan mengerjakan:

1. Membuat menu navigasi di bagian bawah layar dengan `BottomNavigationView`.
2. Memakai Fragment sebagai halaman utama aplikasi, lengkap dengan Toolbar dan View Binding.
3. Menampilkan daftar data dengan `ListView`, `ArrayAdapter`, dan `SimpleAdapter`.
4. Membuat tampilan daftar sendiri (gambar, nama, pesan) dengan Custom Adapter dan Glide.

YANG PERLU DISIAPKAN

- Project Pertemuan 6 sudah selesai dan bisa dijalankan (Splash → Login → Main, serta `SixthActivity` dengan tiga Fragment).
- Izin INTERNET sudah ada di manifest (dari Pertemuan 5), karena kita akan memuat gambar dari internet.
- Emulator atau HP Android punya koneksi internet.

ALUR MODUL

Tahap	Yang dikerjakan	Isi praktikum	Hasil akhir
0	Persiapan project	Struktur folder dan `BaseActivity`	Project siap
1	Bottom Navigation	Menu, layout, `BaseActivity`, `replaceFragment()`	Tiga tab bisa berpindah
2	Fragment sebagai halaman utama	Home, Toolbar, View Binding, alur ke `BaseActivity`	Home menggantikan `MainActivity`
3	ListView	`ArrayAdapter` dan `SimpleAdapter` di tab More	Daftar bisa di-scroll dan diklik
4	Custom Adapter	Model, layout item, Glide, `MessageAdapter` di tab Message	Daftar dengan foto, nama, dan pesan
5	Latihan mandiri	4 soal latihan	Tugas dikumpulkan

CARA MENGIKUTI MODUL

Penanda	Arti
Langkah bernomor	Kerjakan berurutan di Android Studio, jangan melompat.
CEK HASIL	Daftar singkat di akhir tiap tahap. Kalau semuanya sesuai, baru lanjut.
PENTING	Aturan inti yang wajib diingat saat mengerjakan.
Stabilo kuning	Istilah atau baris kode yang wajib dipahami.

---

0 Persiapan Project

Kita lanjutkan project dari Pertemuan 6, jadi **tidak perlu membuat project baru**. Pada pertemuan ini aplikasi kita punya satu halaman utama baru, `BaseActivity`, yang isinya berganti-ganti lewat Fragment. Halaman-halaman dari pertemuan lalu dikumpulkan di dalam package `Home`.

0.1 Susun package dan activity baru

1. Buat `BaseActivity` di package utama, sejajar dengan `MainActivity` (New → Activity → Empty Views Activity).
2. Buat tiga package baru bernama `Home`, `Message`, dan `More`.
3. Di dalam masing-masing package, buat Fragment kosong: `HomeFragment`, `MessageFragment`, dan `MoreFragment` (New → Fragment → Fragment (Blank)).
4. Pindahkan semua package `pertemuan_2` sampai `pertemuan_6` ke dalam package `Home` dengan cara menariknya (drag) ke `Home`, lalu klik Refactor.

```
com.example.irgi_tic
  Home
    pertemuan_2 … pertemuan_6
    HomeFragment
  Message
    MessageFragment
  More
    MoreFragment
  BaseActivity
  LoginActivity
  MainActivity
  SplashScreenActivity
res
  layout
    activity_base.xml
    fragment_home.xml
    fragment_message.xml
    fragment_more.xml
    item_message.xml
  menu
    bottom_nav_menu.xml
    main_menu.xml
  drawable
    ic_home.xml
    ic_message.xml
    ic_more.xml
    ic_arrow_back.xml
```

CATATAN File berlatar warna adalah file baru di pertemuan ini. `MessageModel` dan `MessageAdapter` di package `Message` baru kita buat di Tahap 4. `MainActivity` masih ada sekarang, dan baru dihapus di Tahap 2.

PERHATIAN Setelah memindahkan package, buka `AndroidManifest.xml` dan pastikan nama activity-nya ikut berubah (misalnya `.Home.pertemuan_5.LimaActivity`). Android Studio biasanya mengubahnya otomatis. Kalau ada yang merah, perbaiki namanya.

0.2 Jadikan BaseActivity halaman pertama (sementara)

Supaya mudah dicoba, kita lewati dulu Splash dan Login. Di `AndroidManifest.xml`, pindahkan `intent-filter` MAIN/LAUNCHER dari `SplashScreenActivity` ke `BaseActivity`.

XML

```xml
<activity
    android:name=".BaseActivity"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

CATATAN Ini hanya sementara. Di akhir Tahap 2, LAUNCHER kita kembalikan ke `SplashScreenActivity`.

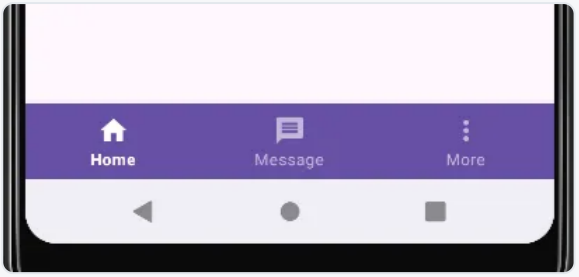
CEK HASIL **Sebelum lanjut, cek dulu:**

- Struktur folder sama seperti gambar di atas.
- Aplikasi bisa dijalankan tanpa error dan langsung membuka `BaseActivity` (masih kosong).

---

1 Bottom Navigation

TUJUAN Membuat `BaseActivity` dengan Bottom Navigation tiga menu (Home, Message, More) yang mengganti Fragment saat ditekan.



Hasil akhir: Bottom Navigation dengan tiga menu

1.1 Buat tiga Fragment sederhana

Seperti di Pertemuan 6, sisakan hanya `onCreateView()` di masing-masing Fragment. Contoh untuk `HomeFragment`:

KOTLIN

```kotlin
class HomeFragment : Fragment() {

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        return inflater.inflate(R.layout.fragment_home, container, false)
    }
}
```

Lalu desain layout masing-masing dengan teks sederhana dulu: “Ini Fragment Home”, “Ini Fragment Message”, dan “Ini Fragment More” (cara seperti `fragment_satu.xml` di Pertemuan 6). `MessageFragment` dan `MoreFragment` kodenya sama, hanya nama layout-nya berbeda.

1.2 Siapkan ikon dan resource menu

Buat tiga Vector Asset di `res/drawable` bernama `ic_home`, `ic_message`, dan `ic_more` (klik kanan drawable → New → Vector Asset, lalu cari ikon yang cocok). Setelah itu buat resource menu: klik kanan folder `res/menu` → New → Menu Resource File, beri nama `bottom_nav_menu`.

```xml
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/home"
        android:icon="@drawable/ic_home"
        android:title="Home" />
    <item
        android:id="@+id/message"
        android:icon="@drawable/ic_message"
        android:title="Message" />
    <item
        android:id="@+id/more"
        android:icon="@drawable/ic_more"
        android:title="More" />
</menu>
```

1.3 Susun activity_base.xml

Isi layout dengan `FrameLayout` sebagai container dan `BottomNavigationView` di bagian bawah.

```xml
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".BaseActivity">

    <FrameLayout
        android:id="@+id/fragment_container"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        app:layout_constraintBottom_toTopOf="@+id/bottom_nav_view"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/bottom_nav_view"
        style="@style/Widget.MaterialComponents.BottomNavigationView.Colored"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:layout_constraintBottom_toBottomOf="parent"
        app:menu="@menu/bottom_nav_menu" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

1.4 Uji klik menu di BaseActivity

Sebelum menampilkan Fragment, kita pastikan dulu tiap menu bisa diklik. Tambahkan di `onCreate()` pada `BaseActivity`:

```kotlin
class BaseActivity : AppCompatActivity() {
    private lateinit var binding: ActivityBaseBinding

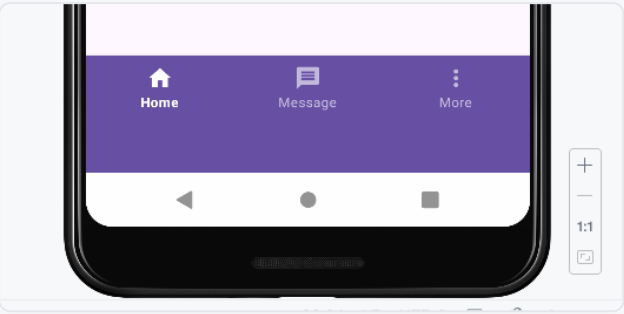
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityBaseBinding.inflate(layoutInflater)
```

```
        setContentView(binding.root)
        ...
        binding.bottomNavView.setOnItemSelectedListener {
            when (it.itemId) {
                R.id.home -> {
                    Toast.makeText(this, "Home Clicked", Toast.LENGTH_SHORT).show()
                    true
                }
                R.id.message -> {
                    Toast.makeText(this, "Message Clicked", Toast.LENGTH_SHORT).show()
                    true
                }
                R.id.more -> {
                    Toast.makeText(this, "More Clicked", Toast.LENGTH_SHORT).show()
                    true
                }
                else -> false // false jika item tidak dikenal
            }
        }
    }
}
```

Jalankan aplikasinya. Bottom Navigation harus tampil, dan setiap ikon memunculkan Toast ketika diklik.

Kalau ada jarak kosong di bawah Bottom Navigation

Pada sebagian tampilan, Bottom Navigation terlihat "terangkat" karena ada jarak kosong di bawahnya.



Bottom Navigation dengan jarak kosong di bagian bawah

Itu terjadi karena kode bawaan menambahkan *padding* untuk system bar di sisi bawah. Ubah nilai `systemBars.bottom` menjadi `0` pada baris insets di `onCreate()` `BaseActivity`:

KOTLIN

```
ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
    val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
    v.setPadding(systemBars.left, systemBars.top, systemBars.right, 0) // bottom = 0
    insets
}
```

1.5 Ganti Toast dengan replaceFragment()

Sekarang ganti Toast tadi dengan perpindahan Fragment. Tambahkan fungsi `replaceFragment()` di bawah `onCreate()`:

```kotlin
private fun replaceFragment(fragment: Fragment) {
    supportFragmentManager.beginTransaction()
        .replace(binding.fragmentContainer.id, fragment)
        // .addToBackStack(null) sengaja dinonaktifkan: saat back langsung keluar aplikasi
        .commit()
}
```

Lalu sesuaikan `onCreate()` supaya `HomeFragment` tampil sebagai Fragment awal:

```kotlin
// HomeFragment sebagai fragment default
replaceFragment(HomeFragment())

binding.bottomNavView.setOnItemSelectedListener {
    when (it.itemId) {
        R.id.home -> {
            replaceFragment(HomeFragment())
            true
        }
        R.id.message -> {
            replaceFragment(MessageFragment())
            true
        }
        R.id.more -> {
            replaceFragment(MoreFragment())
            true
        }
        else -> false
    }
}
```

PERHATIAN Import `Fragment` dari `androidx.fragment.app.Fragment`, dan import Fragment dari package `Home`, `Message`, serta `More` dengan Alt + Enter.

CEK HASIL **Sebelum lanjut, cek dulu:**

- Tiga menu muncul di bagian bawah dan tidak ada jarak kosong di bawahnya.
- `HomeFragment` tampil saat aplikasi dibuka.
- Menekan Message dan More mengganti isi layar, sementara Bottom Navigation tetap di tempat.
- Tombol back dari tab mana pun langsung keluar dari aplikasi.

2 Fragment sebagai Halaman Utama

TUJUAN Menjadikan `HomeFragment` pengganti `MainActivity`, lengkap dengan Toolbar, View Binding, dan alur Splash → Login → `BaseActivity`.

PENTING **Ingat poin-poin ini:**

- Binding di Fragment: `_binding` dibuat di `onCreateView()`, dipakai di `onViewCreated()`, dikosongkan di `onDestroyView()`.
- Di dalam Fragment, `this` diganti `requireContext()`.
- Toolbar dipasang lewat `(requireActivity() as AppCompatActivity).setSupportActionBar(...)`.

2.1 Ubah fragment_home.xml

Ubah `FrameLayout` menjadi `ConstraintLayout`. Hapus `TextView` "Ini Fragment Home", lalu isi dengan Toolbar, teks Welcome, dan semua tombol dari `activity_main.xml` (Pertemuan 2–6 dan Logout). Salin saja komponen-komponennya dari `activity_main.xml`, lalu sesuaikan constraint-nya.

XML

```
<androidx.constraintlayout.widget.ConstraintLayout ...
    tools:context=".Home.HomeFragment">

    <androidx.appcompat.widget.Toolbar
        android:id="@+id/toolbar"
        android:layout_width="0dp"
        android:layout_height="?attr/actionBarSize"
        android:background="?attr/colorPrimary"
        app:titleTextColor="@android:color/white"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <TextView
        android:id="@+id/tvWelcome"
        android:text="Welcome!"
        ... />

    <Button android:id="@+id/btnToSecond" ... />
    <Button android:id="@+id/btnToThird" ... />
    <Button android:id="@+id/btnToFourth" ... />
    <Button android:id="@+id/btnToLima" ... />
    <Button android:id="@+id/btnToSixth" ... />
    <Button android:id="@+id/btnLogout" ... />
</androidx.constraintlayout.widget.ConstraintLayout>
```

CATATAN Constraint tombol paling atas diarahkan ke bawah Toolbar (`layout_constraintTop_toBottomOf="@+id/toolbar"`), dan tombol berikutnya ke bawah tombol sebelumnya, seperti di `activity_main.xml`.

2.2 Sesuaikan HomeFragment

Ganti `onCreateView()` dengan versi binding, tambahkan `onViewCreated()` dan `onDestroyView()`, lalu salin pemanggilan tombol dari `MainActivity`.

KOTLIN

```kotlin
class HomeFragment : Fragment() {
    private var _binding: FragmentHomeBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentHomeBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        (requireActivity() as AppCompatActivity).setSupportActionBar(binding.toolbar)
        (requireActivity() as AppCompatActivity).supportActionBar?.title = "Home"

        binding.btnToSecond.setOnClickListener { ... }
        binding.btnToThird.setOnClickListener { ... }
        binding.btnToFourth.setOnClickListener { ... }
        binding.btnToLima.setOnClickListener {
            startActivity(Intent(requireContext(), LimaActivity::class.java))
        }
        binding.btnToSixth.setOnClickListener {
            startActivity(Intent(requireContext(), SixthActivity::class.java))
        }
        // btnLogout ada di langkah berikutnya
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

Pada kode tombol yang disalin, ganti `this` menjadi `requireContext()` supaya tidak error.

KOTLIN

```kotlin
// Sebelum
val intent = Intent(this, SecondActivity::class.java)

// Sesudah
val intent = Intent(requireContext(), SecondActivity::class.java)
```

Sekarang tombol Logout. Sama seperti sebelumnya, tapi setelah “Ya” kita kembali ke `LoginActivity` dan membersihkan seluruh back stack (cara dari Latihan Pertemuan 6).

KOTLIN

```kotlin
binding.btnLogout.setOnClickListener {
    val sharedPref = requireContext().getSharedPreferences("user_pref", Context.MODE_PRIVATE)

    AlertDialog.Builder(requireContext())
        .setTitle("Logout")
        .setMessage("Apakah Anda yakin ingin keluar?")
        .setPositiveButton("Ya") { dialog, _ ->
```

```
            sharedPref.edit().clear().apply()
            dialog.dismiss()

            val intent = Intent(requireContext(), LoginActivity::class.java)
            intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
            startActivity(intent)
        }
        .setNegativeButton("Batal") { dialog, _ -> dialog.dismiss() }
        .show()
}
```

2.3 Pasang Toolbar di MessageFragment dan MoreFragment

Terapkan cara yang sama di dua Fragment lainnya: ubah `FrameLayout` menjadi `ConstraintLayout`, tambahkan Toolbar dengan id `toolbar` (tidak perlu `CoordinatorLayout` dulu), lalu pasang binding seperti `HomeFragment`. Contoh untuk `MessageFragment`:

KOTLIN

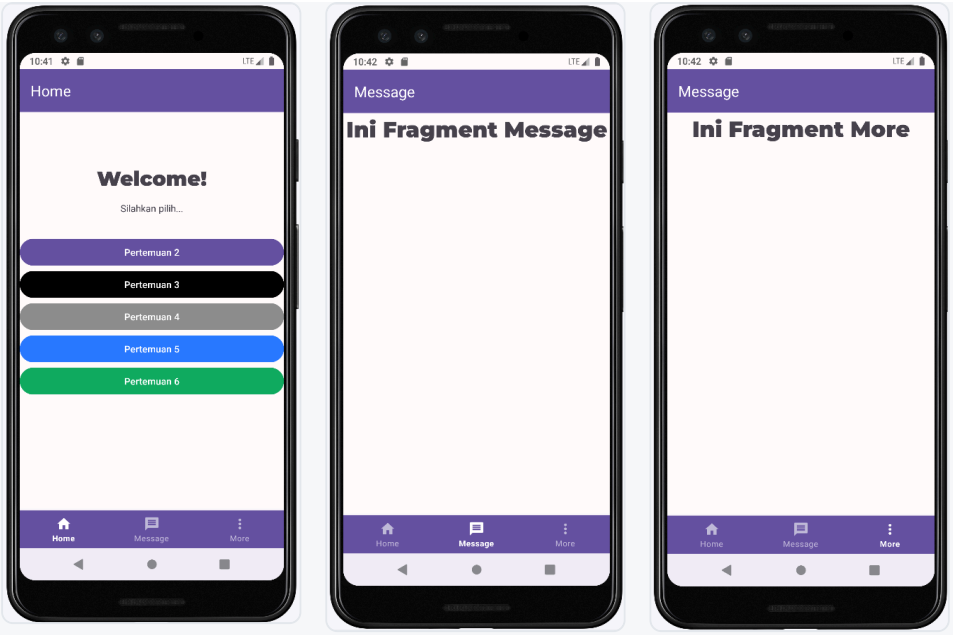
```kotlin
class MessageFragment : Fragment() {
    private var _binding: FragmentMessageBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentMessageBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        (requireActivity() as AppCompatActivity).setSupportActionBar(binding.toolbar)
        (requireActivity() as AppCompatActivity).supportActionBar?.title = "Message"
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

Untuk `MoreFragment`, ganti `FragmentMessageBinding` dengan `FragmentMoreBinding`, dan judul Toolbar menjadi “More”.



Home Message More

2.4 Jadikan BaseActivity halaman utama

1. Hapus `MainActivity` beserta `activity_main.xml` (klik kanan → Refactor → Safe Delete).
2. Di `AndroidManifest.xml`, kembalikan `intent-filter` MAIN/LAUNCHER ke `SplashScreenActivity`, dan hapus dari `BaseActivity`.
3. Di `SplashScreenActivity`, arahkan ke `BaseActivity`.
4. Di `LoginActivity`, arahkan juga ke `BaseActivity` setelah login berhasil.

KOTLIN

```kotlin
// SplashScreenActivity
val target = if (isLogin) BaseActivity::class.java else LoginActivity::class.java
startActivity(Intent(this@SplashScreenActivity, target))
finish()

// LoginActivity, saat login berhasil
startActivity(Intent(this, BaseActivity::class.java))
finish()
```

PERHATIAN Jika setelah menghapus `MainActivity` masih ada kode yang merah karena merujuk ke `MainActivity` (misalnya di halaman pertemuan lain), ganti dengan `BaseActivity` atau cukup `finish()` sesuai kebutuhan.

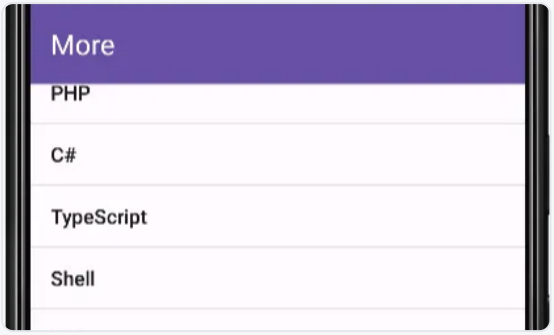
CEK HASIL **Sebelum lanjut, cek dulu:**

- Alur Splash → Login → `BaseActivity` berjalan, dan jika sudah login langsung ke `BaseActivity`.
- Tab Home menampilkan tombol Pertemuan 2–6, dan setiap tombol bisa membuka halamannya.
- Tab Home, Message, dan More masing-masing punya Toolbar dengan judulnya sendiri.
- Logout membawa kembali ke halaman login, dan membuka ulang aplikasi tetap menampilkan login.

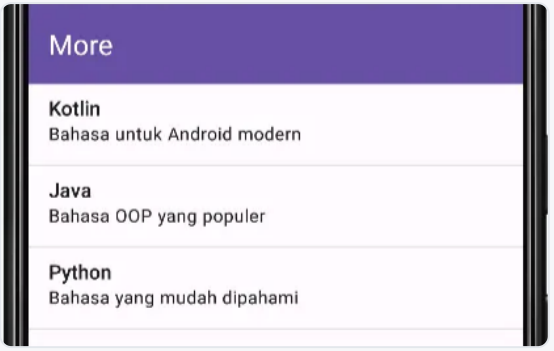
3 ListView

TUJUAN Menampilkan daftar bahasa pemrograman di tab More dengan `ListView`, memakai `ArrayAdapter` lalu `SimpleAdapter`.

Layout bawaan	Isi per baris	Adapter yang dipakai
`simple_list_item_1`	1 teks	`ArrayAdapter`
`simple_list_item_2`	2 teks: judul dan deskripsi	`SimpleAdapter`



simple_list_item_1



simple_list_item_2

3.1 Susun fragment_more.xml

Ubah layout induk menjadi `CoordinatorLayout`, bungkus Toolbar dengan `AppBarLayout`, lalu tambahkan `ListView`.

XML

```xml
<androidx.coordinatorlayout.widget.CoordinatorLayout ...
    tools:context=".More.MoreFragment">

    <com.google.android.material.appbar.AppBarLayout
        android:id="@+id/appBarLayout"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:theme="@style/ThemeOverlay.AppCompat.Dark.ActionBar">

        <androidx.appcompat.widget.Toolbar
            android:id="@+id/toolbar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
            android:background="?attr/colorPrimary"
            app:layout_scrollFlags="scroll|enterAlways"
            app:titleTextColor="@android:color/white" />
    </com.google.android.material.appbar.AppBarLayout>

    <ListView
        android:id="@+id/listViewItems"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        app:layout_behavior="@string/appbar_scrolling_view_behavior" />
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

3.2 Siapkan data di MoreFragment

Tambahkan daftar data di dalam class `MoreFragment`, di atas `onCreateView()`.

KOTLIN

```kotlin
private val dataList = listOf(
    "Kotlin", "Java", "Python", "C++", "JavaScript",
    "Dart", "Swift", "Go", "Ruby", "R",
    "PHP", "C#", "TypeScript", "Shell", "SQL",
    "Perl", "Rust", "Scala", "Haskell", "Lua",
    "Erlang", "Prolog", "Assembly", "Objective-C", "VBA"
)
```

3.3 Pasang ArrayAdapter (simple_list_item_1)

Di `onViewCreated()` pada `MoreFragment` (setelah kode Toolbar), buat adapter, hubungkan ke `ListView`, lalu tambahkan aksi saat item diklik.

KOTLIN

```kotlin
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)
    // ... kode Toolbar ...

    // Adapter: penghubung dataList dengan layout simple_list_item_1
    val adapter = ArrayAdapter(
        requireContext(),
        android.R.layout.simple_list_item_1,
        dataList
    )

    // Hubungkan listViewItems dengan adapter
    binding.listViewItems.adapter = adapter

    // Aksi saat item diklik
    binding.listViewItems.setOnItemClickListener { _, _, position, _ ->
        val selectedItem = dataList[position]
        Toast.makeText(requireContext(), "Kamu memilih: $selectedItem",
Toast.LENGTH_SHORT).show()
    }
}
```

Jalankan aplikasi, buka tab More, lalu scroll dan klik beberapa item. `simple_list_item_1` hanya menampilkan satu teks per baris.

3.4 Pasang SimpleAdapter (simple_list_item_2)

Jika ingin menambah deskripsi di bawah judul, pakai `SimpleAdapter` dengan `simple_list_item_2`. Tambahkan data baru di bawah `dataList`:

KOTLIN

```kotlin
private val dataListWithDesc = listOf(
    mapOf("title" to "Kotlin", "desc" to "Bahasa untuk Android modern"),
    mapOf("title" to "Java", "desc" to "Bahasa OOP yang populer"),
    mapOf("title" to "Python", "desc" to "Bahasa yang mudah dipahami")
)
```

Lalu ubah adapter dan listener di `onViewCreated()` menjadi:

```kotlin
val adapter = SimpleAdapter(
    requireContext(),
    dataListWithDesc,
    android.R.layout.simple_list_item_2,
    arrayOf("title", "desc"),                       // key yang diambil dari data
    intArrayOf(android.R.id.text1, android.R.id.text2) // tempat tampilnya
)

binding.listViewItems.adapter = adapter

binding.listViewItems.setOnItemClickListener { _, _, position, _ ->
    val selectedItem = dataListWithDesc[position]
    val title = selectedItem["title"]
    val desc = selectedItem["desc"]
    Toast.makeText(requireContext(), "Kamu memilih: $title ($desc)",
Toast.LENGTH_SHORT).show()
}
```

CATATAN `arrayOf("title", "desc")` adalah key pada data, dan `intArrayOf(text1, text2)` adalah tempat tampilnya. Urutan keduanya harus berpasangan: `title` tampil di `text1`, `desc` di `text2`.

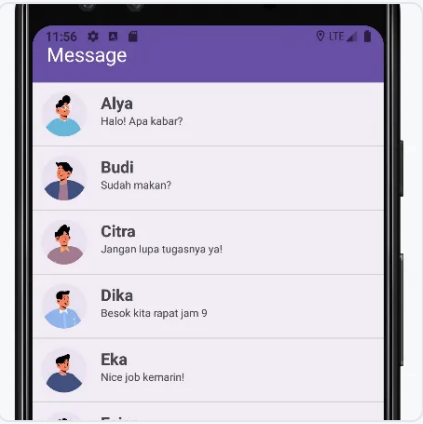
CEK HASIL

Sebelum lanjut, cek dulu:

- Tab More menampilkan daftar yang bisa di-scroll, dan Toolbar ikut bergulir.
- Klik item memunculkan Toast “Kamu memilih: …”.
- Setelah memakai `SimpleAdapter`, tiap item menampilkan judul dan deskripsi.

4 ListView dengan Custom Adapter

TUJUAN Membuat daftar pesan dengan foto, nama, dan isi pesan di tab Message memakai Custom Adapter dan Glide.



Hasil akhir: daftar pesan dengan foto, nama, dan isi pesan

File yang dibuat	Peran
`MessageModel.kt`	Wadah data satu item: `senderName`, `messageText`, `avatarUrl`
`item_message.xml`	Tampilan satu baris: foto, nama, pesan
`MessageAdapter.kt`	Mengisi layout item dengan data tiap baris lewat `getView()`
Glide (`build.gradle`)	Memuat gambar dari URL ke `ImageView`

PENTING **Ingat poin-poin ini:**

- `getView()` dipanggil untuk tiap baris, di sanalah data dimasukkan ke layout.
- Aksi klik item dipasang di adapter, bukan di Fragment.
- Setelah menambah Glide, wajib klik Sync Now.

4.1 Tambahkan Glide

Buka `build.gradle` (Module: app), tambahkan satu baris di dalam `dependencies`, lalu klik **Sync Now**.

KOTLIN

```
dependencies {
    ...
    implementation("com.github.bumptech.glide:glide:4.16.0")
}
```

PERHATIAN Jangan lupa Sync Now. Ini yang paling sering terlewat. Pastikan juga izin INTERNET masih ada di `AndroidManifest.xml` (dari Pertemuan 5).

4.2 Buat MessageModel

Klik kanan package `Message` → New → Kotlin Class/File, pilih **Data class**, beri nama `MessageModel`.

```kotlin
data class MessageModel(
    val senderName: String,
    val messageText: String,
    val avatarUrl: String
)
```

Karena kita menampilkan nama, pesan, dan gambar, modelnya punya tiga atribut. Kalau nanti butuh data lain (misalnya waktu), tinggal tambah atributnya.

4.3 Desain item_message.xml

Klik kanan `res/layout` → New → Layout Resource File, beri nama `item_message`. Isi dengan kode berikut.

```xml
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:padding="20dp">

    <ImageView
        android:id="@+id/avatarImg"
        android:layout_width="50dp"
        android:layout_height="50dp"
        android:contentDescription="Foto pengirim"
        android:scaleType="centerCrop"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <TextView
        android:id="@+id/textSender"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="16dp"
        android:text="Item Name"
        android:textSize="18sp"
        android:textStyle="bold"
        app:layout_constraintStart_toEndOf="@+id/avatarImg"
        app:layout_constraintTop_toTopOf="parent" />

    <TextView
        android:id="@+id/textMessage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="16dp"
        android:layout_marginTop="10dp"
        android:text="Item Desc"
        android:textSize="14sp"
        app:layout_constraintStart_toEndOf="@+id/avatarImg"
        app:layout_constraintTop_toBottomOf="@+id/textSender" />
</androidx.constraintlayout.widget.ConstraintLayout>
```



Pratinjau item_message.xml

CATATAN Perhatikan id `avatarImg`, `textSender`, dan `textMessage`. Ketiganya dipakai di `MessageAdapter`.

4.4 Tambahkan ListView di fragment_message.xml

Seperti di tab More, ubah `fragment_message.xml` menjadi `CoordinatorLayout` dengan `AppBarLayout` untuk Toolbar, lalu tambahkan `ListView` dengan id `listMessageItems`.

XML

```xml
<androidx.coordinatorlayout.widget.CoordinatorLayout ...>

    <com.google.android.material.appbar.AppBarLayout
        android:id="@+id/appBarLayout"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:theme="@style/ThemeOverlay.AppCompat.Dark.ActionBar">
        <!-- Toolbar id "toolbar", sama seperti di fragment_more.xml -->
    </com.google.android.material.appbar.AppBarLayout>

    <ListView
        android:id="@+id/listMessageItems"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        app:layout_behavior="@string/appbar_scrolling_view_behavior" />
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

4.5 Siapkan data pesan di MessageFragment

Tambahkan di dalam class `MessageFragment`:

KOTLIN

```kotlin
private val messageList = listOf(
    MessageModel("Alya", "Halo! Apa kabar?", "https://avatar.iran.liara.run/public/1"),
    MessageModel("Budi", "Sudah makan?", "https://avatar.iran.liara.run/public/2"),
    MessageModel("Citra", "Jangan lupa tugasnya ya!",
"https://avatar.iran.liara.run/public/3"),
    MessageModel("Dika", "Besok kita rapat jam 9", "https://avatar.iran.liara.run/public/4"),
    MessageModel("Eka", "Nice job kemarin!", "https://avatar.iran.liara.run/public/5"),
    MessageModel("Fajar", "Lagi ngapain?", "https://avatar.iran.liara.run/public/6"),
    MessageModel("Gita", "Boleh minta tolong?", "https://avatar.iran.liara.run/public/7"),
    MessageModel("Hana", "Lihat email ya", "https://avatar.iran.liara.run/public/8"),
    MessageModel("Irfan", "Oke noted", "https://avatar.iran.liara.run/public/9"),
    MessageModel("Joko", "Sampai jumpa besok", "https://avatar.iran.liara.run/public/10")
)
```

4.6 Buat MessageAdapter

Klik kanan package `Message` → New → Kotlin Class/File, pilih **Class**, beri nama `MessageAdapter`.

KOTLIN

```kotlin
class MessageAdapter(
    context: Context,
    private val messages: List<MessageModel>
```

```
) : ArrayAdapter<MessageModel>(context, 0, messages) {

    override fun getView(position: Int, convertView: View?, parent: ViewGroup): View {
        val binding = ItemMessageBinding.inflate(LayoutInflater.from(context), parent, false)
        val view = binding.root
        val data = messages[position]

        // Muat gambar dari URL ke ImageView
        Glide.with(context)
            .load(data.avatarUrl)
            .into(binding.avatarImg)

        binding.textSender.text = data.senderName
        binding.textMessage.text = data.messageText

        return view
    }
}
```

Adapter ini menghubungkan data (`MessageModel`) dengan tampilan satu baris (`item_message.xml`). Untuk tiap baris, `getView()` mengambil data pada posisi tersebut lalu mengisi nama, pesan, dan gambarnya.

4.7 Pasang adapter di onViewCreated

KOTLIN

```
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    super.onViewCreated(view, savedInstanceState)
    // ... kode Toolbar ...

    val adapter = MessageAdapter(requireContext(), messageList)
    binding.listMessageItems.adapter = adapter
}
```

Jalankan aplikasi dan buka tab Message. Daftar pesan dengan foto, nama, dan isi pesan harus tampil.

4.8 Aksi klik pada setiap item

Karena adapter yang mengurus tiap baris, pasang aksi klik di bagian akhir `getView()`, tepat sebelum `return view`.

KOTLIN

```
view.setOnClickListener {
    Snackbar.make(
        parent,
        "Pesan dari ${data.senderName}: ${data.messageText}",
        Snackbar.LENGTH_SHORT
    ).show()
}
return view
```

PERHATIAN Import `Snackbar` dari `com.google.android.material.snackbar.Snackbar`. Kalau gambar tidak muncul, periksa koneksi internet, izin INTERNET di manifest, dan pastikan Glide sudah di-sync. Alamat gambar berasal dari layanan luar; jika layanannya sedang bermasalah, ganti dengan alamat gambar lain.

CEK HASIL **Sebelum lanjut, cek dulu:**

- Tab Message menampilkan 10 pesan lengkap dengan foto, nama, dan isi pesan.
- Daftar bisa di-scroll dan Toolbar ikut bergulir.
- Klik pada satu pesan memunculkan Snackbar “Pesan dari ... : ...”.

---

5 Latihan Mandiri

Sekarang giliran kalian mengembangkan sendiri aplikasinya.

5.1 Sapaan pengguna di Home

WAJIB

Lanjutan dari latihan Pertemuan 6: tampilkan “Halo, (username)” pada `tvWelcome` di `HomeFragment`, dengan membaca key `username` dari SharedPreferences. Ingat, di Fragment akses SharedPreferences lewat `requireContext()`.

```kotlin
val sharedPref = requireContext().getSharedPreferences("user_pref", Context.MODE_PRIVATE)
val username = sharedPref.getString("username", "Pengguna")
```

5.2 Lengkapi daftar dengan deskripsi

WAJIB

Pada `dataListWithDesc` di tab More, tambahkan data hingga minimal 8 bahasa pemrograman, masing-masing dengan deskripsi singkat dalam kata-kata kalian sendiri.

5.3 Tambahkan waktu pada pesan

TANTANGAN

Tambahkan atribut `time` (misalnya “09.15”) pada `MessageModel`, tambahkan `TextView` baru di `item_message.xml` untuk menampilkannya di sisi kanan, lalu isi di `MessageAdapter`. Jangan lupa menambahkan nilai `time` pada tiap data di `messageList`.

5.4 Halaman detail pesan

TANTANGAN

Saat sebuah pesan diklik, buka halaman baru (Activity) yang menampilkan nama pengirim, foto, dan isi pesan. Kirim datanya lewat `Intent` dengan `putExtra()` seperti yang pernah kita pakai di pertemuan sebelumnya, dan tambahkan Toolbar dengan tombol back seperti Pertemuan 5.

Checklist pengumpulan

- [ ] Bottom Navigation berfungsi dan tab bisa berpindah tanpa menumpuk back stack
- [ ] Alur Splash → Login → `BaseActivity` berjalan, dan Logout kembali ke halaman login
- [ ] Tab More menampilkan daftar dengan `ArrayAdapter` atau `SimpleAdapter`, tab Message memakai Custom Adapter
- [ ] Latihan 5.1 dan 5.2 selesai

6 Kalau Ada yang Error

Gejala	Kemungkinan penyebab	Solusi
`ActivityBaseBinding` atau `FragmentHomeBinding` merah	Layout belum dibuat atau namanya berbeda, atau belum Sync	Cek nama layout (`activity_base`, `fragment_home`), lalu Sync dan Rebuild
Ada jarak kosong di bawah Bottom Navigation	Padding system bar sisi bawah dari kode bawaan	Ubah `systemBars.bottom` menjadi `0` di `BaseActivity`
Menu ditekan tapi isi tidak berganti	`replaceFragment()` belum dipanggil, atau tidak ada `true` di tiap cabang	Periksa isi `when` pada `setOnItemSelectedListener`
`Intent(this, …)` merah di Fragment	Fragment bukan Context	Ganti `this` menjadi `requireContext()`
Aplikasi berhenti setelah `MainActivity` dihapus	Masih ada kode yang merujuk ke `MainActivity`	Ganti dengan `BaseActivity` atau `finish()`
Aplikasi berhenti dengan pesan binding `null` setelah ganti tab	Binding dipakai setelah `onDestroyView()`	Pakai binding hanya di `onViewCreated()` sampai sebelum `onDestroyView()`
Daftar tidak muncul	Id `ListView` di layout dan di kode berbeda, atau `layout_behavior` terlewat	Samakan id (`listViewItems`, `listMessageItems`) dan cek `layout_behavior`
Gambar avatar tidak muncul	Glide belum di-sync, tidak ada internet, atau izin INTERNET hilang	Sync Now, cek koneksi, dan izin di `AndroidManifest.xml`
Klik item tidak berefek pada Custom Adapter	Listener dipasang di Fragment, bukan di `getView()`	Pasang `view.setOnClickListener` di `MessageAdapter`